Introduction

Why ant robots I

Ant colonies coordinate at scale without any individual ant holding a global view of the colony's state: each ant senses locally, acts locally, and influences its nestmates only indirectly, through shared, persistent traces in the environment — a mode of coordination biologists call *stigmergy*. This is not a decorative metaphor for this template; it is the actual reason the domain was chosen (per the governing ISA's Principles: "the ant-robot domain is chosen because bounded local computation, local sensing, and stigmergic … coordination are actually true of ant colonies and actually motivate per-agent local DB + local networking — the domain must earn its place scientifically, not decoratively"). A decentralized multiagent simulation with **no shared global state** — each agent owns its own database file and its own network endpoint — is a direct

computational analogue of that biological constraint, and it happens to be exactly the setting where illegal-state bugs (a corrupted message reaching the wrong agent's database, a half-finished handshake silently treated as complete) are both easy to introduce and easy to make structurally impossible.

Why strong typing is the research subject, not an implementation detail I

The public exemplars in this monorepo (`projects/templates/*`) cover numerical analysis, prose composition, textbooks, and Active Inference modeling, but none of them makes strongly-typed program design the load- bearing research claim. This gap has a specific character: Python is gradually, optionally, structurally typed — `mypy --strict` can reject a genuinely broad class of illegal programs at edit-time and in CI, but Python has no compiler-enforced affine or structural-linearity discipline and no dependent-type checker in its standard toolchain. A template that wants to *demonstrate* strong typing honestly therefore has to draw a line between what the type checker actually proves and what remains a runtime discipline that a determined caller could still violate — and then back every claim on

either side of that line with a test that would fail if the claim were false. sec. 3 states that line explicitly; sec. 3.6 pins each strong claim in it to a numbered test.

Reader's guide to the manuscript I

Reader's guide to the manuscript I

- **sec. 3** walks the ADT (`Result[T, E]`), nominal-ID (`AgentId`/`MessageId`/`TxnId`), session-typed protocol (`IdleSession`/`HandshakingSession`/`EstablishedSession`/`ClosedSession`), and affine-discipline (`TransactionHandle`) layers, each grounded in Wadler (2015), Pierce (2002), Milner (1978), Honda et al. (1998), and Jung et al. (2018), and closes with sec. 3.6 — the explicit proof-scoping section.
- **sec. 4** frames the per-agent SQLite schema as a functor $\mathrm{Schema} \to \mathbf{Set}$, per Fong and Spivak (2018) and Spivak and Kent (2012), and states plainly that this is a design lens, not a machine-checked functoriality proof.
- **sec. 5** frames each agent's per-tick decision as an approximate expected-free-energy minimization, citing Friston (2005), and connects it to collective biological organization via the Memory Evolutive Systems framework of (Ehresmann and Vanbremeersch 2007).
- **sec. 6** reports the multi-agent colony integration test's stigmergic convergence result (honestly scoped as a real mechanism, not yet a claim of emergence), the mypy-oracle proof-of-detection results, and discusses what evidence would be needed to widen any of these claims.
- **sec. 7** points to the full bibliography.

Related empirical grounding I

Two further citations anchor the paper's claim that static typing has measurable, non-hypothetical value, and that value has documented limits: Gao et al. (2017) quantified detectable bug categories in a large corpus of JavaScript projects and found a meaningful — but bounded — fraction of real-world bugs are the kind a type system would have caught; the Google Security Blog's 2024 retrospective (Google Security Blog (2024)) reports a measured drop in memory-safety vulnerabilities in newly-written, memory-safe-by-default Android code, evidence from a production codebase rather than a benchmark suite. Neither claim licenses an inference that *any* particular typed discipline in this template's code prevents *any* particular class of bug beyond what its own paired test demonstrates — they establish only that the general research question (does typing prevent real bugs?)

has real, cited, non-anecdotal answers in the literature this template draws on.

Finally, Ongaro and Ousterhout (2014) (Raft) is cited not because this template implements distributed consensus — it explicitly does not (see `Out of Scope`: "No production consensus/Raft/Paxos implementation") — but because the colony's typed shared "pheromone field" substrate (`src/template_formal/colony/pheromone.py`) is a deliberately minimal, documentary stand-in for the kind of shared-coordination-state problem Raft solves in full; the comparison is drawn explicitly, not left implicit, in sec. 6.